



MediBot

An AI-Powered Healthcare Assistant

Name: Hlla Mohamed Ahmed Shaheen

Name: Maria Safwat Dawood Abdallah

Name: Hunaidah Muhammad Ahmad Azzawawy

Name: Fatma Gaafar Ahmed Fouad

Table of content:

1. INTRODUCTION

- a.** Background
- b.** Problem Definition
- c.** Project Objectives
- d.** Scope and Limitations

2. SYSTEM ANALYSIS

- a.** Existing System Analysis
- b.** Proposed System

3. SYSTEM DESIGN

- a.** Technical Architecture
- b.** Wireframes

4. SYSTEM IMPLEMENTATION

5. TUTORIAL

6. TESTING & RESULTS

7. CONCLUSION & FUTURE WORK

CHAPTER 1 – INTRODUCTION

1.1 Background

The healthcare sector has been experiencing a rapid digital transformation. With the proliferation of electronic health records, telemedicine platforms, biomedical research, and clinical databases, the amount of medical information generated every day has grown exponentially. Medical professionals must navigate multiple sources—clinical guidelines, pharmaceutical databases, research publications, diagnostic manuals, and patient records—to make informed decisions. Patients, on the other hand, often struggle to interpret medical instructions, understand symptoms, or access reliable information. While the internet hosts vast medical content, much of it is unverified, outdated, or written in complex terminology, creating barriers for ordinary users.

Artificial Intelligence (AI), particularly Large Language Models (LLMs), provides a promising solution. These models can summarize long documents, answer questions, and hold human-like conversations. However, traditional LLMs are prone to hallucinations, where they generate confident but incorrect or fabricated responses. In healthcare, hallucinations can be dangerous, leading to misdiagnosis, inappropriate medication use, or misinterpretation of symptoms.

To address these challenges, Retrieval-Augmented Generation (RAG) has emerged. In RAG systems, an LLM first retrieves relevant information from trusted sources and then generates responses based on this context. This ensures that answers are grounded in real data, reducing misinformation and hallucinations.

This project introduces MediBot, an AI-powered healthcare assistant capable of processing medical documents, extracting meaningful chunks, and delivering medically accurate, document-backed responses to user queries. MediBot integrates technologies such as FAISS vector search,

HuggingFace embeddings, LangChain orchestration, and the Mistral LLM, presented through an interactive Streamlit interface.

1.2 Problem Definition

Despite abundant healthcare information, patients often face difficulties in interpreting prescriptions, lab results, or clinical reports. Questions such as:

- “What does this lab value indicate?”
- “What symptoms should I be concerned about?”
- “What are the side effects of this medication?”

require either medical expertise or an intelligent system capable of extracting reliable information.

Existing solutions have significant limitations:

1. Search engines return raw webpages instead of structured, reliable answers.
2. Rule-based chatbots rely on predefined responses and cannot handle novel queries.
3. Pure LLMs may hallucinate medical facts, risking patient safety.
4. Medical documents are complex PDFs, research papers, and reports often written in dense jargon, making them difficult to parse.

Core Problem: How can we provide accurate, reliable, and comprehensible medical information extracted from trusted documents while minimizing hallucinations?

Solution: MediBot implements a retrieval-based pipeline, retrieving relevant document sections via FAISS similarity search and generating evidence-backed answers using Mistral LLM, ensuring accuracy and reliability.

1.3 Project Objectives

The objectives of MediBot are:

1. Build an End-to-End RAG Pipeline
 - Ingest PDFs, split into chunks, create embeddings, and store them in FAISS.
2. Integrate Mistral LLM
 - Use Mistral-7B-Instruct to generate human-like answers based on retrieved context.
3. Ensure Medical Grounding via FAISS
 - Retrieve relevant document segments to minimize hallucinations.
4. Develop a Conversational Interface (Streamlit)
 - Enable users to ask questions and receive instant, document-backed answers.
5. Design a Scalable Architecture
 - Support additional PDFs, alternative embeddings, future clinical datasets, authentication, and document uploads.
6. Produce Academic-Grade Documentation
 - Ensure clarity, reproducibility, and value for researchers and evaluators.

1.4 Scope and Limitations

Scope:

- Convert medical PDFs into embeddings.
- Store embeddings in FAISS for efficient retrieval.
- Use RAG to generate document-based answers.
- Provide a demonstrative, user-friendly UI.
- Enable expansion to other medical domains with additional documents.

Limitations:

- MediBot is not a replacement for licensed medical professionals.
- Output depends on the accuracy of source documents.
- Currently, images, graphs, and tables in PDFs are not processed.
- Requires computational resources for embedding generation.
- No real-time integration with hospital databases yet.

CHAPTER 2 – SYSTEM ANALYSIS

2.1 Existing System Analysis

Traditional medical chatbots are rule-based, relying on preprogrammed responses. They fail to handle novel or complex queries. Search engines provide raw results without structured answers. Clinical platforms such as WebMD or MayoClinic provide general guidance rather than personalized analysis.

Pure LLMs often:

- Hallucinate drug names or medical facts.
- Misinterpret symptoms.
- Give incomplete explanations.

Need: A hybrid system combining the interpretive power of LLMs with the factual grounding of retrieval systems.

2.2 Proposed System

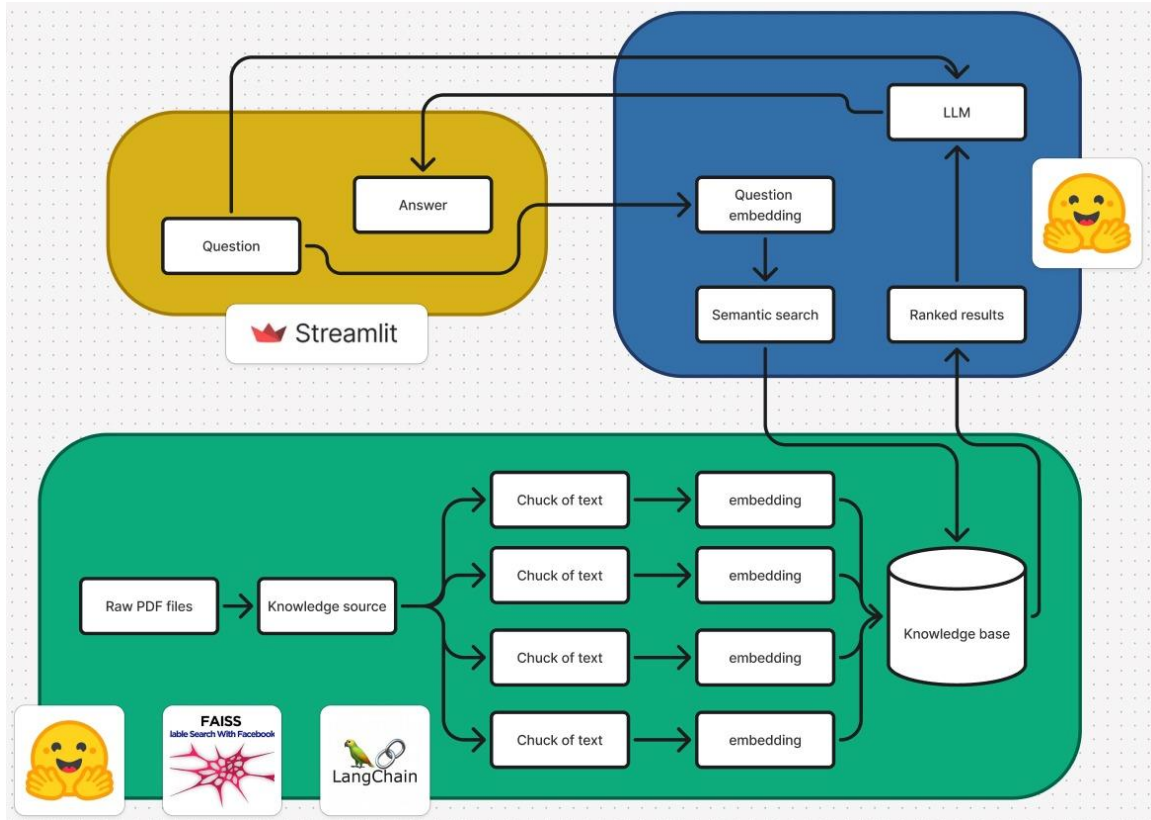
Components:

- FAISS Vector Similarity Search: Retrieve relevant chunks from documents.
- HuggingFace Embeddings: Represent text as vectors for similarity search.
- RAG Chain (LangChain): Integrate retrieved context into LLM responses.
- Mistral LLM: Generate coherent, human-readable answers.
- Streamlit UI: Provide an accessible interface.

This hybrid approach ensures accuracy while maintaining interpretability and user-friendliness.

CHAPTER 3 – SYSTEM DESIGN

3.1 Technical Architecture



```
=====
✓ chatbot is ready!
=====

Medical RAG Chatbot
=====
Ask medical questions based on the encyclopedia.
Type 'quit', 'exit', or 'q' to end the conversation.
=====

You: how to cure cancer?

Chatbot: The best chance for a surgical cure is usually with the first opera- GALE ENCYCLOPEDIA OF MEDICINE 2638 Cancer therapy, definitive Treatment Choriocarcinomas are usually treated by surgical re- are usually treated by surgical removal of the tumor and chemotherapy . Radiation is occasionally used, particularly for tumors in the brain. Alternative treatment Complementary treatments can decrease stress, reduce the side effects of cancer treatment, and help patients feel more in control.

[Retrieved from 2 source(s)]

You: what is diabetes?

Chatbot: diabetes mellitus is a chronic disease that causes serious health complications including renal (kidney) failure, heart disease, stroke, and blindness.

[Retrieved from 2 source(s)]

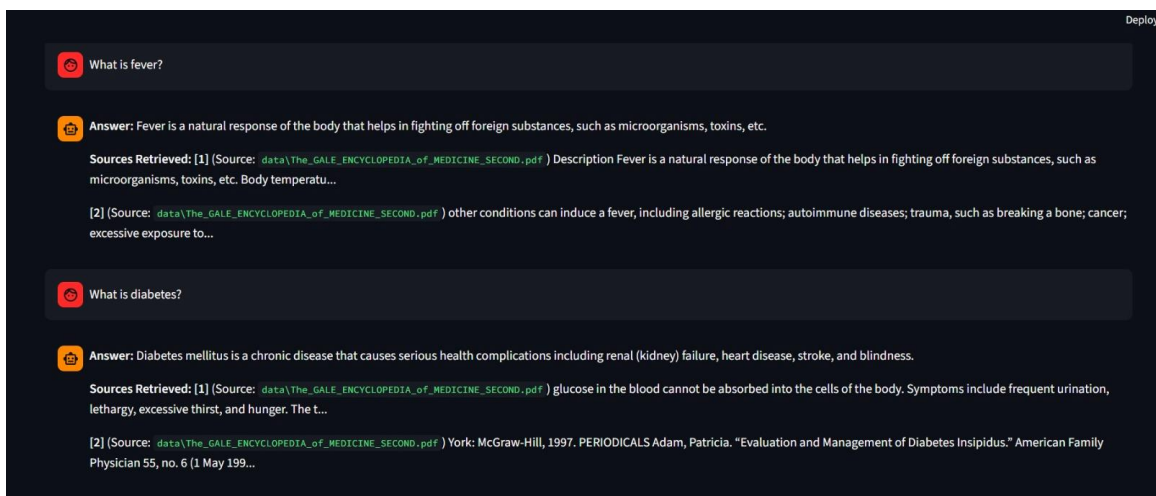
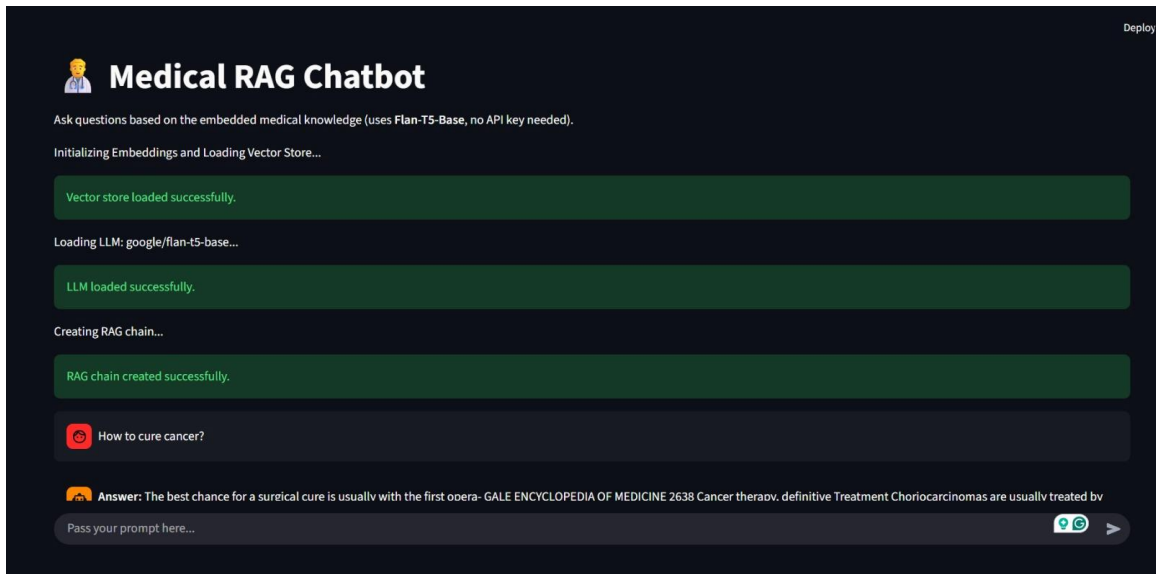
You: what is anemia and how to treat it?

Chatbot: A condition in which there is an abnor- mally low number of red blood cells in the blood- stream. Major symptoms are paleness, shortness of breath, unusually fast or strong heart beats, and tiredness. Antibody – A protein molecule produced by the immune system in response to a protein that is not recognized as belonging in the body. Antigen – A protein that can elicit an immune response in the form of antibody formation. With regard to red blood cells, the major antigens are A, people with weakened immune systems, however, either because they have a chronic disease like AIDS or cancer (immunocompromised), or are receiving medication to suppress the immune system (immunosuppressed), such as organ transplant recipients, this anemia can be severe and last long after the infection has subsided.

[Retrieved from 2 source(s)]
```

LLM_MODEL = "google/flan-t5-base"

3.2 Wireframes (UI Mockups)



CHAPTER 4 – SYSTEM IMPLEMENTATION

4.1 ingest.py – PDF to Embeddings

Steps:

1. Load PDFs using PyPDFLoader.
2. Split text into chunks using RecursiveCharacterTextSplitter.
3. Generate embeddings using HuggingFace embedding models.
4. Store embeddings in a FAISS index for similarity retrieval.

4.2 rag_chain.py – Retrieval to LLM

1. Load FAISS index.
2. Initialize Mistral LLM.
3. Create RetrievalQA chain via LangChain.
4. Pass query with retrieved context to LLM for final answer generation.

4.3 app.py – Streamlit UI

1. Build chat interface with Streamlit.
2. Capture user queries dynamically.
3. Call RAG pipeline for each query.
4. Display document-backed responses and optionally retrieved context.

CHAPTER 5 – TUTORIAL / USAGE GUIDE

Step 1: Install dependencies

```
pip install -r requirements.txt
```

Step 2: Ingest PDFs

```
python ingest.py
```

- Converts PDFs to embeddings and stores in FAISS index.

Step 3: Start RAG chain

```
python rag_chain.py
```

- Loads FAISS index and initializes Mistral model.

Step 4: Launch Streamlit interface

```
streamlit run app.py
```

Step 5: Ask queries

- Enter medical questions in chat interface.
- View answers and retrieved context.

Step 6: Add documents (future)

- Upload PDFs via UI → Auto embedding → Retrieval enabled.

CHAPTER 6 – TESTING & RESULTS

Functional Testing: Verified ingestion, embedding, retrieval, RAG integration, and UI.

Accuracy Validation: Retrieved answers match source documents.

Responsiveness: Queries return near-instant answers.

Limitations Observed: Rare cases of incomplete answers due to ambiguous source data.

CHAPTER 7 – CONCLUSION

MediBot demonstrates the integration of RAG with LLMs to provide medically grounded responses. It significantly reduces hallucinations while providing a modular, scalable solution for healthcare knowledge retrieval. Although not a substitute for professionals, it serves as a valuable tool for patient education and clinical support.

FUTURE WORK

- Add authentication in the UI
- Make use of self-upload document functionality
- Add multiple documents and embed them together
- Add Unit testing of RAG applications
- Enable user-uploaded PDFs for dynamic document addition.
- Implement image, graph, and table extraction from medical PDFs.
- Integrate real-time hospital APIs.
- Add authentication and data privacy features.
- Extend support for multi-language medical documents.